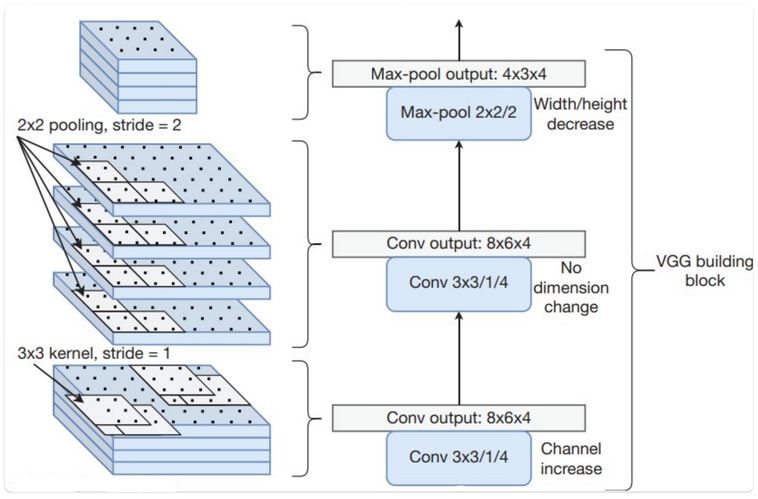
3. VGGNet:

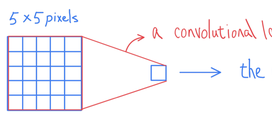
A. they use a fixed kernel size of 3×3, a stride of 1, and an appropriate padding mechanism in each of convolutional layers.
when using a stride of 1 with a proper padding mechanism, the width and height dimensions of each feature map in the next convolutional layer remain identical to the width and height dimensions of each feature map in the current convolutional layer.
this makes it possible to make a VGGNet arbitrarily deeper without running into the problem that the width and height dimensions of feature maps in the convolutional layers deep in a neuron network become to small.

B. we want each neuron to identify larger size features by hierarchically combining smaller size features.
the VGGNet solves this by using the 2×2/2 max pooling operation between groups of convolutional layers.

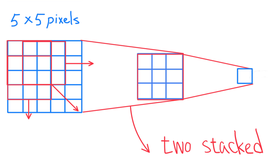
C. a typical VGG module in a VGGNet is a group of convolutional layers with fixed kernel size of 3×3 and a stride of 1, which is also followed by a 2×2/2 max pooling operation.




D. the kernel size 3×3 in a convolutional layer is relatively small.
however, if we can combine a group of these convolutional layers together into a VGG module,
the last convolutional layer of these convolutional layers will not only possess the same receptive field as a single convolutional layer with larger kernel size into a VGG module
but also use fewer unique weights than a single convolutional layer with larger kernel size into a VGG module.
for example:
the second convolutional layer of a group of two convolutional layers with 3×3 kernel size into a VGG module
will not only have the same receptive field as a single convolutional layer with 5×5 kernel size into a VGG module
but also use (3×3)×(Cout)×2, [Cout = num of the output channels in each convolutional layer] unique weights than a single convolutional layer with 5×5 kernel size into a VGG module's (5×5)×(Cout)×1, [Cout = num of the output channels in a convolutional layer] unique weights.

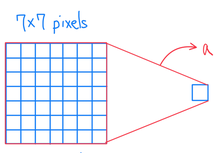



→ the receptive field of a single convolutional layer with 5×5 kernel size into a VGG module = 5×5 pixels.

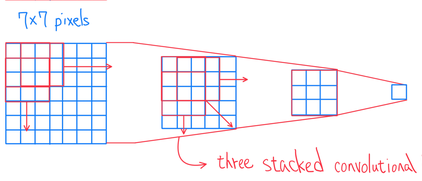



→ the receptive field of the second convolutional layer of a group of two convolutional layers with 3×3 kernel size into a VGG module = 5×5 pixels.

for example:
the third convolutional layer of a group of three convolutional layers with 3×3 kernel size into a VGG module
will not only have the same receptive field as a single convolutional layer with 7×7 kernel size into a VGG module
but also use (3×3)×(Cout)×3, [Cout = num of the output channels in each convolutional layer] unique weights than a single convolutional layer with 7×7 kernel size into a VGG module's (7×7)×(Cout)×1, [Cout = num of the output channels in a convolutional layer] unique weights.




→ the receptive field of a single convolutional layer with 7×7 kernel size into a VGG module = 7×7 pixels.




→ the receptive field of the third convolutional layer of a group of three convolutional layers with 3×3 kernel size into a VGG module = 7×7 pixels.

E. the convolutional layers of first VGG module in a VGGNet all start with 64 output channels.
while using a 2×2/2 max pooling operation, the width and height dimensions of input data for the next convolutional layer are halved.
the subsequent convolutional layers will keep doubling their num of output channels until their num of output channels is capped at 512.
although the width and height dimensions of input data for the next convolutional layer are still halved after a 2×2/2 max pooling operation, the num of subsequent convolutional layers' output channels are kept constant at 512.

F. each neuron of all convolutional layers in a VGGNet use ReLU as an activation function.

G. the key result of a VGGNet study indicates that the accuracy did increase with a model depth increasing up to 16 convolutional layers,
but the accuracy flats out to about the same accuracy for a model with 19 convolutional layers after a model depth increasing over 16 convolutional layers,

H.

| 13 WEIGHT LAYERS | 16 WEIGHT LAYERS |
|---|---|
| Input RGB image (224×224×3) | |
| Conv 3×3/1/64 | Conv 3×3/1/64 |
| **Conv 3×3/1/64** | Conv 3×3/1/64 |
| 2×2/2 max pooling | |
| Conv 3×3/1/128 | Conv 3×3/1/128 |
| **Conv 3×3/1/128** | Conv 3×3/1/128 |
| 2×2/2 max pooling | |
| Conv 3×3/1/256 | Conv 3×3/1/256 |
| Conv 3×3/1/256 | Conv 3×3/1/256 |
| | **Conv 1×1/1/256** |
| 2×2/2 max pooling | |
| Conv 3×3/1/512 | Conv 3×3/1/512 |
| Conv 3×3/1/512 | Conv 3×3/1/512 |
| | **Conv 1×1/1/512** |
| 2×2/2 max pooling | |
| Conv 3×3/1/512 | Conv 3×3/1/512 |
| Conv 3×3/1/512 | Conv 3×3/1/512 |
| | **Conv 1×1/1/512** |
| 2×2/2 max pooling | |
| Fully connected, 4,096 | |
| Fully connected, 4,096 | |
| Fully connected, 1,000 with softmax | |